

Illusions: Hierarchical Modeling Tutorial

Mahbod Mehrvarz

August 2025

This note accompanies *Localizing Structure in Individual Differences: A Visual Illusion Case Study* (Mehrvarz, Popat, Rouder). It shows how to fit two hierarchical models to the illusions data: a Gaussian baseline with an inverse-Wishart prior and a hierarchical Bayesian factor model.

Overview

Goal: fit models that separate trial noise from across-individual structure.

1. load and standardize data
2. Gaussian baseline with IW prior (population vs individual correlations)
3. hierarchical factor model with simple identification
4. posterior summaries, tables, and plots

Packages: R2jags, abind, infinitefactor, rlist, RColorBrewer.

```
set.seed(42)
options(width = 90)
need = c("R2jags", "abind", "infinitefactor", "rlist", "RColorBrewer", "corrplot")
to_install = setdiff(need, rownames(installed.packages()))
if (length(to_install) > 0) install.packages(to_install, quiet = TRUE)
library(R2jags); library(abind); library(infinitefactor); library(rlist)
library(RColorBrewer)
```

Data

```

url1 = "https://raw.githubusercontent.com/spec1/ind-illusions/"
url2 = "refs/heads/public/_data/clean_data.csv"
url = paste0(url1, url2)
dat = read.csv(url)

dat$S = as.numeric(factor(dat$sub))
dat$J0 = as.numeric(factor(dat$task))      # 1..5 tasks
dat$JV = (dat$J0 - 1) * 2 + dat$version    # 1..10 task×version

# variance-normalize by task×version
v1 = tapply(dat$y, list(dat$S, dat$JV), var)
divisor = sqrt(apply(v1, 2, mean))
dat$yc = dat$y / divisor[dat$JV]

I = length(unique(dat$S))
J = 10                                # task×version
D = 7                                # 5 task factors + 2 general
N = nrow(dat)

task_names = c("Brentano 1", "Brentano 2", "Ebbinghaus 1", "Ebbinghaus 2",
               "Poggendorf 1", "Poggendorf 2", "Ponzo 1", "Ponzo 2",
               "Zoellner 1", "Zoellner 2")

```

Gaussian baseline with inverse-Wishart prior (before factors)

We fit a hierarchical Gaussian that captures two correlation sources.

For y_{ijk} with person i , indicator j (task×version), replicate k :

$$\begin{aligned}
y_{ijk} \mid \theta_{ij}, \tau_j^2 &\sim \mathcal{N}(\theta_{ij}, \tau_j^2), \\
\theta_i \mid \boldsymbol{\mu}, \boldsymbol{\Omega} &\sim \mathcal{N}_J(\boldsymbol{\mu}, \boldsymbol{\Omega}^{-1}).
\end{aligned}$$

Priors:

$$\mu_j \sim \mathcal{N}(\mu_m, \mu_s^2), \quad \tau_j^{-2} \sim \text{Gamma}(0.5, 0.5), \quad \boldsymbol{\Omega} \sim \mathcal{W}_J(\nu, \mathbf{T}),$$

with $\nu = J + 1$ and $\mathbf{T} = \text{diag}(t^2)$.

We report posterior correlations at two levels:

- **population:** from $\Sigma = \Omega^{-1}$
- **individual:** from θ_{-i} within each draw

```

modIWJags = "
model{
  for (i in 1:I){
    theta[i,1:J] ~ dmnorm(mu[], pSig2[,])
  }
  for (j in 1:J){
    mu[j] ~ dnorm(mu_m, pow(mu_s, -2))
    pTau2[j] ~ dgamma(0.5, 0.5)
  }
  pSig2 ~ dwish(diagJ * pow(tuneT, 2), J + 1)

  for (n in 1:N){
    center[n] <- theta[sub[n], task[n]]
    y[n] ~ dnorm(center[n], pTau2[task[n]])
  }
}
"

runJags = function(mod, dat, pars, nchains = 2, niter = 3000, nburnin = 1000){
  jags(model.file = textConnection(mod),
    data = dat, n.chains = nchains, n.iter = niter,
    n.burnin = nburnin, n.thin = 1, parameters.to.save = pars)
}

iw_data = list(
  y = dat$yc,
  task = dat$JV,
  sub = dat$S,
  I = I,
  J = J,
  N = N,
  diagJ = diag(J),
  tuneT = 1,
  mu_m = 0,
  mu_s = 2
)

fitIW = runJags(modIWJags, iw_data, c("theta", "pSig2"))

```

```

## Compiling model graph
## Resolving undeclared variables

```

```
##      Allocating nodes
## Graph information:
##      Observed stochastic nodes: 20547
##      Unobserved stochastic nodes: 159
##      Total graph size: 63296
##
## Initializing model

drawsIW = fitIW$BUGSoutput$sims.list
thetaIW = drawsIW$theta           #  $M \times I \times J$ 
pSig2IW = drawsIW$pSig2          #  $M \times J \times J$ 
M_IW = dim(thetaIW)[1]
```

Correlations and intervals.

```
Ybar = tapply(dat$yc, list(dat$S, dat$JV), mean)
colnames(Ybar) = task_names
samp_cor = cor(Ybar); dimnames(samp_cor) = list(task_names, task_names)

pop_list = vector("list", M_IW)
for (m in 1:M_IW){
  Sigma = solve(pSig2IW[m,,])
  pop_list[[m]] = cov2cor(Sigma)
}

ind_list = vector("list", M_IW)
for (m in 1:M_IW){
  Th = thetaIW[m,,]
  colnames(Th) = task_names
  ind_list[[m]] = cor(Th)
}

summarize_corrs = function(L){
  A = abind::abind(L, along = 3)
  mean = apply(A, c(1,2), mean)
  lo    = apply(A, c(1,2), quantile, probs = .025)
  hi    = apply(A, c(1,2), quantile, probs = .975)
  dimnames(mean) = dimnames(lo) = dimnames(hi) = dimnames(samp_cor)
  list(mean = mean, lo = lo, hi = hi)
}

pop_sum = summarize_corrs(pop_list)
ind_sum = summarize_corrs(ind_list)
```

```

pairs_df = {
  idx = which(upper.tri(samp_cor), arr.ind = TRUE)
  data.frame(
    pair = paste0(rownames(samp_cor)[idx[,1]], "-", colnames(samp_cor)[idx[,2]]),
    samp = samp_cor[idx],
    pop_mean = pop_sum$mean[idx],
    pop_lo = pop_sum$lo[idx],
    pop_hi = pop_sum$hi[idx],
    ind_mean = ind_sum$mean[idx],
    ind_lo = ind_sum$lo[idx],
    ind_hi = ind_sum$hi[idx]
  )
}

# knitr::kable(
#   pairs_df,
#   align = "c",
#   caption =
#     "Off-diagonal correlations: sample vs model (posterior means and 95% CIs).",
#   digits = 3
# )

head(pairs_df)

```

```

##           pair      samp  pop_mean  pop_lo  pop_hi  ind_mean
## 1  Brentano 1-Brentano 2 0.247005779 0.269612241 0.09361552 0.4331707 0.273739963
## 2  Brentano 1-Ebbinghaus 1 0.005250349 0.008829498 -0.17481222 0.1943226 0.007406104
## 3  Brentano 2-Ebbinghaus 1 0.102127721 0.117887597 -0.07083169 0.2969332 0.119869737
## 4  Brentano 1-Ebbinghaus 2 0.178387663 0.197782224 0.01593790 0.3741468 0.200421871
## 5  Brentano 2-Ebbinghaus 2 0.115683054 0.134520836 -0.05658050 0.3142843 0.133785055
## 6 Ebbinghaus 1-Ebbinghaus 2 0.413071119 0.477075388 0.31184627 0.6191364 0.484665329
##      ind_lo  ind_hi
## 1 0.20321705 0.34625707
## 2 -0.08175769 0.09658122
## 3 0.02901426 0.21078965
## 4 0.11922384 0.27838325
## 5 0.04708789 0.21837447
## 6 0.39376282 0.57066175

```

Plots. Heatmaps and interval comparison.

```

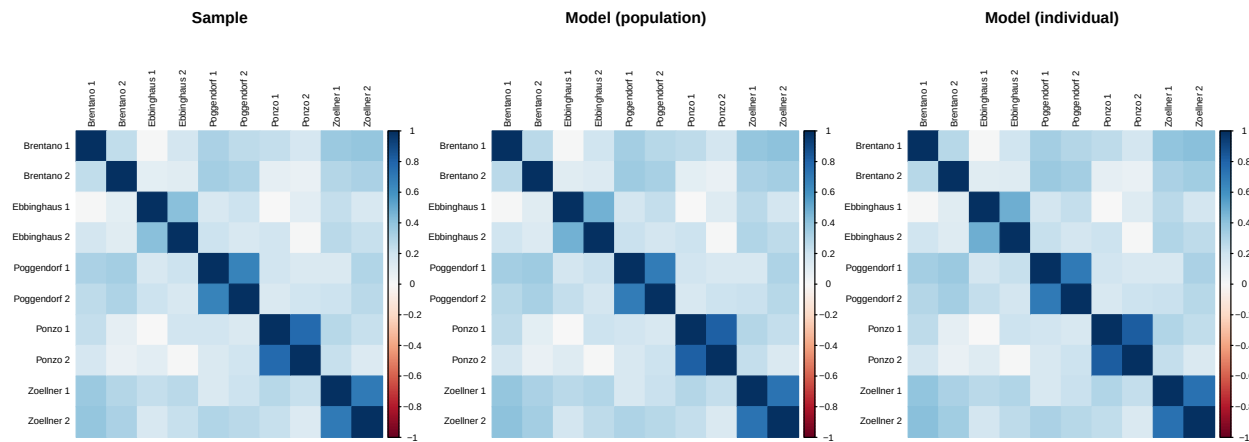
cols = colorRampPalette(RColorBrewer::brewer.pal(11, "RdBu"))(201)
require(corrplot)

```

```

par(mfrow = c(1,3), mar = c(1.5,1.5,2,1))
corrplot(samp_cor,
  method = "color", col = cols, cl.lim = c(-1,1),
  tl.col = "black", tl.cex = .7, addgrid.col = NA, title = "Sample",
  mar = c(0,0,1,0))
corrplot(pop_sum$mean,
  method = "color", col = cols, cl.lim = c(-1,1),
  tl.col = "black", tl.cex = .7, addgrid.col = NA,
  title = "Model (population)", mar = c(0,0,1,0))
corrplot(ind_sum$mean,
  method = "color", col = cols, cl.lim = c(-1,1),
  tl.col = "black", tl.cex = .7, addgrid.col = NA,
  title = "Model (individual)", mar = c(0,0,1,0))

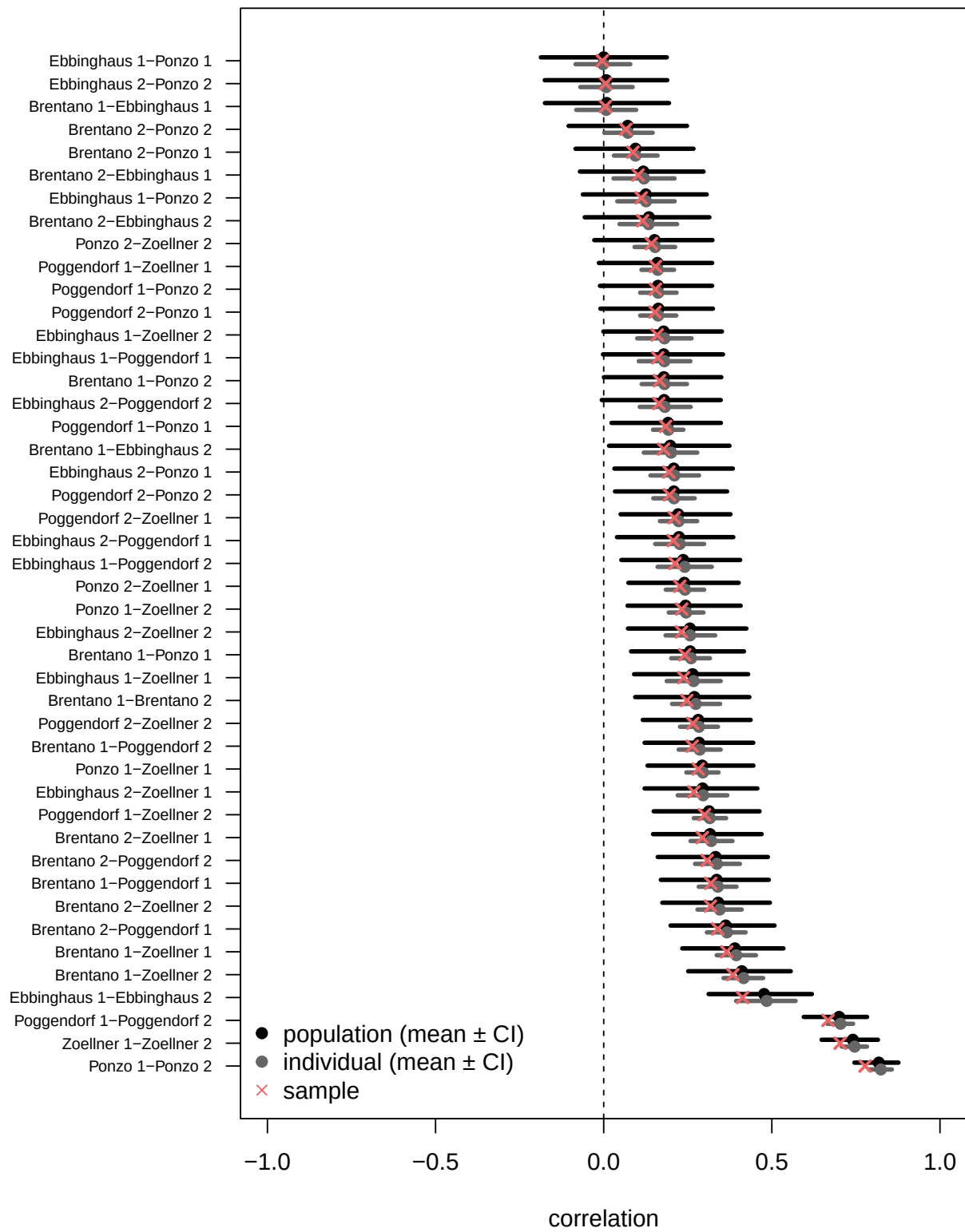
```



```

ord = order(pairs_df$pop_mean, decreasing = TRUE)
y = seq_along(ord)
op = par(mar = c(4, 8.5, 2, 1))
plot(NA, xlim = c(-1, 1), ylim = c(0.5, length(y) + 0.5),
  xlab = "correlation", ylab = "", yaxt = "n")
abline(v = 0, lty = 2, lwd = 1)
segments(pairs_df$pop_lo[ord], y + .15, pairs_df$pop_hi[ord], y + .15, lwd = 3)
segments(pairs_df$ind_lo[ord], y - .15,
  pairs_df$ind_hi[ord], y - .15, lwd = 3, col = "gray40")
points(pairs_df$pop_mean[ord], y + .15, pch = 19)
points(pairs_df$ind_mean[ord], y - .15, pch = 19, col = "gray40")
points(pairs_df$samp[ord], y, pch = 4, cex = 1, lwd = 2, col = "indianred2")
axis(2, at = y, labels = pairs_df$pair[ord], las = 2, cex.axis = 0.7)
legend("bottomleft",
  legend = c("population (mean ± CI)", "individual (mean ± CI)", "sample"),
  pch = c(19, 19, 4), col = c("black", "gray40", "indianred2"),
  pt.cex = c(1, 1, 0.9), bty = "n")

```



par(op)

Hierarchical Bayesian factor model

Measurement model for indicator j and person i :

$$\begin{aligned} y_{ijk} \mid \theta_{ij}, \tau_j^2 &\sim \mathcal{N}(\theta_{ij}, \tau_j^2), \\ \theta_{ij} \mid \mu_j, \boldsymbol{\lambda}_j, \delta_j^2, \boldsymbol{\eta}_i &\sim \mathcal{N}(\mu_j + \boldsymbol{\lambda}_j' \boldsymbol{\eta}_i, \delta_j^2), \\ \boldsymbol{\eta}_i &\sim \mathcal{N}_D(\mathbf{0}, \mathbf{I}_D). \end{aligned}$$

Priors are weak normals on loadings and means, gammas on precisions. Identification uses constraint matrices: five task factors load on their two versions, two general factors are free.

```
mod_bhfm = "  
model {  
  for (j in 1:J) {  
    pTau2[j] ~ dgamma(0.5, 0.5)  
    mu[j] ~ dnorm(mu.m, pow(mu.s, -2))  
    pDel2[j] ~ dgamma(0.5, 0.5 * pow(tuneDelta, 2))  
    del2[j] <- 1 / pDel2[j]  
  
    for (d in 1:D) {  
      lambda_pos[j,d] ~ dnorm(0, pow(tuneLambda, -2)) T(0,)  
      lambda_free[j,d] ~ dnorm(0, pow(tuneLambda, -2))  
      aux_pos[j,d] <- constraint_zero[j,d] - constraint_pos[j,d]  
      lambda[j,d] <- constraint_pos[j,d]*lambda_pos[j,d] + aux_pos[j,d]*lambda_free[j,d]  
    }  
  }  
  
  for (i in 1:I) {  
    for (d in 1:D) { eta[i,d] ~ dnorm(0, 1) }  
  }  
  
  for (i in 1:I) {  
    for (j in 1:J) {  
      center_theta[i,j] <- mu[j] + inprod(lambda[j,1:D], eta[i,1:D])  
      theta[i,j] ~ dnorm(center_theta[i,j], pDel2[j])  
    }  
  }  
}
```



```

for (n in 1:N) {
  center[n] <- theta[sub[n], task[n]]
  y[n] ~ dnorm(center[n], pTau2[task[n]])
}
}
"

```

Constraints.

```

constraint_matrix_zero = matrix(0, nrow = 10, ncol = 7)
for (i2 in 1:5) constraint_matrix_zero[(i2*2-1):(i2*2), i2] = 1
constraint_matrix_zero[, 6:7] = 1

constraint_matrix_pos = matrix(0, nrow = 10, ncol = 7)
for (i2 in 1:5) constraint_matrix_pos[(i2*2-1), i2] = 1

constraint_matrix_zero; constraint_matrix_pos

```

```

##      [,1] [,2] [,3] [,4] [,5] [,6] [,7]
## [1,]    1    0    0    0    0    1    1
## [2,]    1    0    0    0    0    1    1
## [3,]    0    1    0    0    0    1    1
## [4,]    0    1    0    0    0    1    1
## [5,]    0    0    1    0    0    1    1
## [6,]    0    0    1    0    0    1    1
## [7,]    0    0    0    1    0    1    1
## [8,]    0    0    0    1    0    1    1
## [9,]    0    0    0    0    1    1    1
## [10,]   0    0    0    0    1    1    1

```

```

##      [,1] [,2] [,3] [,4] [,5] [,6] [,7]
## [1,]    1    0    0    0    0    0    0
## [2,]    0    0    0    0    0    0    0
## [3,]    0    1    0    0    0    0    0
## [4,]    0    0    0    0    0    0    0
## [5,]    0    0    1    0    0    0    0
## [6,]    0    0    0    0    0    0    0
## [7,]    0    0    0    1    0    0    0
## [8,]    0    0    0    0    0    0    0
## [9,]    0    0    0    0    1    0    0
## [10,]   0    0    0    0    0    0    0

```

Fit.

```

data_list = list(
  y = dat$yc, task = dat$JV, sub = dat$S,
  I = I, J = J, N = N, D = D,
  constraint_zero = constraint_matrix_zero,
  constraint_pos = constraint_matrix_pos,
  mu.m = 2, mu.s = 2, tuneDelta = 1, tuneLambda = 1
)
parameters = c("lambda", "theta", "mu", "del2", "pTau2", "eta")
fit = runJags(mod_bhfm, data_list, parameters)

```

```

## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 20547
##   Unobserved stochastic nodes: 2516
##   Total graph size: 67445
##
## Initializing model

```

```
invisible(fit)
```

Diagnostics (selected)

```

sel = c("lambda[2,1]", "lambda[4,2]", "lambda[6,3]", "del2[1]", "pTau2[1]", "mu[1]")
summ = fit$BUGSoutput$summary
keep = summ[sel, c("mean", "sd", "2.5%", "50%", "97.5%", "Rhat", "n.eff")]
knitr::kable(round(keep, 3), align = "c",
              caption = "Selected parameters: posterior summaries and diagnostics")

```

Table 1: Selected parameters: posterior summaries and diagnostics

	mean	sd	2.5%	50%	97.5%	Rhat	n.eff
lambda[2,1]	-0.028	0.250	-0.478	-0.042	0.508	1.075	27
lambda[4,2]	0.438	0.087	0.267	0.441	0.601	1.008	1400
lambda[6,3]	0.613	0.222	-0.093	0.655	0.889	1.126	49
del2[1]	0.443	0.138	0.155	0.457	0.684	1.083	29
pTau2[1]	1.000	0.032	0.938	1.000	1.062	1.001	3500

	mean	sd	2.5%	50%	97.5%	Rhat	n.eff
mu[1]	3.034	0.083	2.866	3.035	3.197	1.001	2600

```
op = par(mfrow = c(3, 2), mar = c(3.2, 3.2, 2, 1))
for (v in sel) try(R2jags::traceplot(fit, varname = v), silent = TRUE)
par(op)
```

Post-processing

```
samples = fit$BUGSoutput$sims.list
lambda = samples$lambda
del2    = samples$del2

makePositive = function(lambda){
  Dloc = dim(lambda)[3]
  dimMean = apply(lambda, 3, mean)
  for (d in 1:Dloc){
    s = ifelse(dimMean[d] == 0, 1, sign(dimMean[d]))
    lambda[, ,d] = s * lambda[, ,d]
  }
  lambda
}

align = function(lambda, I = 0){
  M = dim(lambda)[1]; numFactors = dim(lambda)[3]
  eta = array(0, dim = c(M, I, numFactors))
  aligned = jointRot(lambda = lapply(1:M, function(x) lambda[x,,]),
                     eta = lapply(1:M, function(x) eta[x,,]))
  aperm(abind(aligned$lambda, along = 3), c(3,1,2))
}

f2Eigenmax = function(lambda){
  euler2 = function(theta)
    matrix(c(cos(theta), -sin(theta), sin(theta), cos(theta)), 2, 2, byrow = TRUE)
  obj2 = function(theta, b){
    r = b %*% euler2(theta); apply(r^2, 2, sum)[1]
  }
  M = dim(lambda)[1]; pLam = apply(lambda, 2:3, mean)
  out = lambda
  ang = optimize(obj2, c(-pi, pi), maximum = TRUE, b = pLam)$maximum
  for (m in 1:M) out[m,,] = lambda[m,,] %*% euler2(ang)
```

```

    out
}

standardize_lambda = function(lambda, del2){
  M = dim(lambda)[1]; Jloc = dim(lambda)[2]
  lam_std = lambda; del2_std = del2
  Sigma = array(NA_real_, dim = c(M, Jloc, Jloc))
  Rho = array(NA_real_, dim = c(M, Jloc, Jloc))
  for (m in 1:M){
    S = tcrossprod(lambda[m,,]) + diag(del2[m,], Jloc)
    Sigma[m,,] = S; Rho[m,,] = cov2cor(S)
    sigma = sqrt(diag(S))
    lam_std[m,,] = sweep(lambda[m,,], 1, sigma, "/")
    del2_std[m,] = del2[m,] / (sigma^2)
  }
  list(lambda = lam_std, del2 = del2_std, Sigma = Sigma, rho = Rho)
}

```

Align, standardize, summarize.

```

lamb_g = lambda[, , 6:7]
lamb_aligned = align(lamb_g, I = I)
lamb_aligned = f2Eigenmax(lamb_aligned)
lamb_aligned = makePositive(lamb_aligned)
lambda[, , 6:7] = lamb_aligned

std = standardize_lambda(lambda, del2)
lam_std = std$lambda; del2_std = std$del2

makeTable = function(lambda, del2){
  M = dim(lambda)[1]; Jloc = dim(lambda)[2]; Dloc = dim(lambda)[3]
  lam_sum = matrix(0, Jloc, Dloc)
  uni_prop_task_mat = matrix(0, M, Jloc)
  fac_prop_total_mat = matrix(0, M, Dloc)
  uni_prop_total_vec = numeric(M)
  for (m in 1:M){
    tlam = lambda[m,,]; tdel = del2[m,]
    tot_com_task = rowSums(tlam^2)
    tot_task_j = tot_com_task + tdel
    tot_com_fac = colSums(tlam^2)
    tot_uni = sum(tdel)
    tot_var = sum(tdel + tot_com_task)
    lam_sum = lam_sum + tlam
    uni_prop_task_mat[m,] = tdel / tot_task_j
  }
}

```

```

    fac_prop_total_mat[m,] = tot_com_fac / tot_var
    uni_prop_total_vec[m] = tot_uni / tot_var
  }
  lam_mean = round(lam_sum / M, 3)
  uni_prop_task = round(colMeans(uni_prop_task_mat), 3)
  fac_prop_total = round(colMeans(fac_prop_total_mat), 3)
  uni_prop_total = round(mean(uni_prop_total_vec), 3)
  out = cbind(lam_mean, uni_prop_task)
  out = rbind(out, c(fac_prop_total, uni_prop_total))
  rownames(out) = c(task_names, "Prop. Var.")
  colnames(out) = c(paste0("F", 1:D), "Unique Var.")
  out
}

tab = makeTable(lam_std, del2_std)
knitr::kable(
  tab, align = "c",
  caption = "Posterior mean loadings and variance proportions (standardized).")
)

```

Table 2: Posterior mean loadings and variance proportions (standardized).

	F1	F2	F3	F4	F5	F6	F7	Unique Var.
Brentano 1	0.266	0.000	0.000	0.000	0.000	0.549	-0.068	0.520
Brentano 2	-0.032	0.000	0.000	0.000	0.000	0.486	0.046	0.632
Ebbinghaus 1	0.000	0.620	0.000	0.000	0.000	0.281	0.142	0.425
Ebbinghaus 2	0.000	0.608	0.000	0.000	0.000	0.357	-0.109	0.401
Poggendorf 1	0.000	0.000	0.627	0.000	0.000	0.501	0.123	0.239
Poggendorf 2	0.000	0.000	0.570	0.000	0.000	0.497	0.168	0.303
Ponzo 1	0.000	0.000	0.000	0.831	0.000	0.352	-0.096	0.134
Ponzo 2	0.000	0.000	0.000	0.838	0.000	0.288	0.064	0.164
Zoellner 1	0.000	0.000	0.000	0.000	0.559	0.612	-0.110	0.234
Zoellner 2	0.000	0.000	0.000	0.000	0.510	0.634	-0.088	0.259
Prop. Var.	0.020	0.078	0.079	0.140	0.062	0.234	0.057	0.331

Plots

```
dims = dim(lam_std); M = dims[1]; Jloc = dims[2]; Dloc = dims[3]
fac_prop_total_mat = matrix(0, M, Dloc); uni_prop_total_vec = numeric(M)
for (m in 1:M){
  tlam = lam_std[m,,]; tdel = del2_std[m,]
  fac_prop_total_mat[m,] = colSums(tlam^2) / sum(tdel + rowSums(tlam^2))
  uni_prop_total_vec[m] = sum(tdel) / sum(tdel + rowSums(tlam^2))
}
lam_mean = round(apply(lam_std, c(2,3), mean), 3)
base_colors = RColorBrewer::brewer.pal(5, "Set1")
cols = unlist(lapply(base_colors, function(col) c(adjustcolor(col, 0.6), col)))
task_names2 = c("Brentano", "Ebbinghaus", "Poggendorf", "Ponzo", "Zoellner")

par(mfrow = c(1,3), mar = c(5.2, 2.0, 2.5, 0.5))

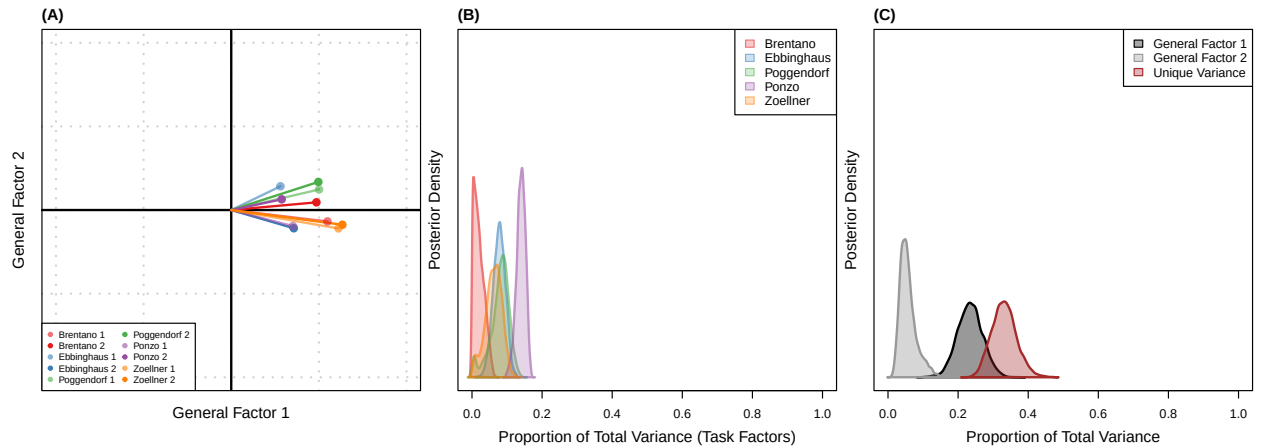
# (A) mean loadings on general factors (F6,F7)
plot(NA, xlim = c(-1, 1), ylim = c(-1, 1), axes = FALSE, xlab = "", ylab = "")
grid(lwd = 2); box(); abline(h = 0, v = 0, lwd = 2)
for (i2 in 1:nrow(lam_mean)) segments(0, 0, lam_mean[i2, 6],
                                     lam_mean[i2, 7], col = cols[i2], lwd = 2)
points(lam_mean[, 6], lam_mean[, 7], col = cols, pch = 19, cex = 1.1, lwd = 2)
legend("bottomleft", legend = task_names, col = cols, pch = 19,
      cex = 0.75, bg = "white", ncol = 2)
mtext("(A)", side = 3, line = 0.4, cex = 0.85, font = 2, adj = 0)
mtext("General Factor 1", side = 1, line = 1.0, cex = 0.8)
mtext("General Factor 2", side = 2, line = 1.0, cex = 0.8)

# (B) variance share per task factor
plot(NA, xlim = c(0, 1), ylim = c(0, 50), axes = FALSE, xlab = "", ylab = "")
axis(1); box()
tcols = cols[c(1,3,5,7,9)]
for (j2 in 1:5){
  dens = density(fac_prop_total_mat[, j2])
  polygon(dens, col = adjustcolor(tcols[j2], 0.4), border = tcols[j2], lwd = 2)
}
legend("topright", legend = task_names2, fill = adjustcolor(tcols, 0.4),
      border = tcols, cex = 0.95)
mtext("(B)", side = 3, line = 0.4, cex = 0.85, font = 2, adj = 0)
mtext("Proportion of Total Variance (Task Factors)",
      side = 1, line = 2.6, cex = 0.8)
mtext("Posterior Density", side = 2, line = 1.0, cex = 0.8)
```

```

# (C) general factors + unique variance
plot(NA, xlim = c(0, 1), ylim = c(0, 50), axes = FALSE, xlab = "", ylab = "")
axis(1); box()
tcols2 = c("black", "gray60", "brown")
for (d2 in 6:7){
  dens = density(fac_prop_total_mat[, d2])
  polygon(dens,
    col = adjustcolor(tcols2[d2-5], 0.4), border = tcols2[d2-5], lwd = 2)
}
dens = density(uni_prop_total_vec)
polygon(dens, col = adjustcolor(tcols2[3], 0.35), border = tcols2[3], lwd = 2)
legend("topright", legend = c("General Factor 1",
  "General Factor 2", "Unique Variance"),
  fill = adjustcolor(tcols2[1:3], 0.35), border = tcols2[1:3], cex = 0.95)
mtext("(C)", side = 3, line = 0.4, cex = 0.85, font = 2, adj = 0)
mtext("Proportion of Total Variance", side = 1, line = 2.6, cex = 0.8)
mtext("Posterior Density", side = 2, line = 1.0, cex = 0.8)

```



End. This file provides the IW baseline and the factor model used in the paper.